

RAPPORT DE PROJET INFORMATIQUE

Screeners Boursiers Technique & IA

Analyse algorithmique d'actifs financiers avec prédiction par apprentissage automatique

Langage	Python 3.10+
Bibliothèques	yfinance, pandas, numpy, PyQt6, matplotlib
Paradigme	Orienté objet (ABC, héritage, polymorphisme)
Année	2025 – 2026

Sommaire

Sommaire	2
Chapitre 1 — Description générale du problème	3
1.1 Introduction — Présentation générale des objectifs	3
1.2 Pistes envisagées.....	3
1.2.1 Approches pour la récupération des données	3
1.2.2 Approches pour le modèle prédictif	4
1.2.3 Approches pour l'interface utilisateur	4
1.3 Hypothèses réductrices	4
Chapitre 2 — Description de l'application	5
2.1 Architecture générale	5
2.2 Principales classes et méthodes	6
2.2.1 ActifFinancier (modeles.py)	9
2.2.2 Hiérarchie Stratégie — héritage et polymorphisme	9
2.2.3 Regression_Ridge (algos_IA.py)	10
2.2.4 Indicateurs pris en compte	11
2.2.5 Fonction récursive — calcul de l'EMA de Wilder	13
2.2.6 News Scraper (algos_IA.py)	13
2.2.7 Interface graphique (app.py)	14
2.2.8 Lancement du programme	14
2.3 Exemple d'utilisation	15
2.3.1 Page d'accueil.....	15
2.3.2 Page principale	16
Chapitre 3 — Conclusion	18
3.1 Tableau récapitulatif des figures imposées	18
3.2 Tests effectués	19
3.3 Limitations identifiées	21
3.4 Apports	22
3.5 Perspectives et améliorations.....	22
3.5.1 Améliorations algorithmiques	22
3.5.2 Améliorations fonctionnelles	23
3.5.3 Améliorations techniques	23

Chapitre 1 — Description générale du problème

1.1 Introduction — Présentation générale des objectifs

Les marchés financiers génèrent chaque jour un volume considérable de données de prix, de volumes et de transactions. Pour un investisseur ou un trader, identifier les opportunités pertinentes parmi des centaines d'actifs est une tâche chronophage qui se prête naturellement à l'automatisation.

Le présent projet a pour ambition de construire un screener boursier — c'est-à-dire un outil de sélection et d'analyse d'actifs — alliant analyse technique classique et intelligence artificielle. Les objectifs sont les suivants :

- Télécharger automatiquement l'historique de cours d'un actif depuis Yahoo Finance.
- Calculer les principaux indicateurs techniques utilisés par les traders (moyennes mobiles, RSI, MACD, volatilité historique, Z-score du volume).
- Backtester plusieurs stratégies algorithmiques et comparer leurs rendements au Buy & Hold de référence.
- Prédire la direction probable du cours du lendemain à l'aide d'un modèle de régression Ridge implémenté de zéro.
- Proposer une interface graphique interactive permettant de visualiser l'ensemble des résultats.

Le projet respecte les contraintes pédagogiques suivantes : programmation orientée objet avec héritage et polymorphisme, structures de données dynamiques, fonctions récursives, lecture/écriture de fichiers, et jeu de tests unitaires.

1.2 Pistes envisagées

Plusieurs approches ont été étudiées avant de retenir l'architecture finale :

1.2.1 Approches pour la récupération des données

- Scraping direct de sites web financiers (Boursorama, Investing.com) : rejeté en raison des restrictions légales et de la fragilité face aux changements de mise en page HTML.
- API payantes (Bloomberg, Refinitiv) : hors de portée du projet pédagogique.
- Bibliothèque yfinance : retenue car gratuite, fiable, et possibilité de récupérer des DataFrames pandas prêts à l'emploi.

1.2.2 Approches pour le modèle prédictif

- Réseaux de neurones récurrents (LSTM) : rejetés car nécessitent TensorFlow/PyTorch et des quantités de données significatives pour converger. Le risque de surapprentissage est élevé sur les horizons courts.
- Forêts aléatoires (Random Forest) : pertinentes mais nécessitent sklearn. Le projet vise une implémentation pédagogique sans boîte noire.
- Régression Ridge manuelle : retenue. La formule analytique exacte est $(X^T X + \alpha I)^{-1} X^T y$, ce qui permet d'implémenter le modèle entièrement avec NumPy et d'en comprendre chaque étape.

1.2.3 Approches pour l'interface utilisateur

- Interface en ligne de commande pure : simple mais peu lisible pour des graphiques.
- Application web (Flask + Chart.js) : nécessite un serveur HTTP, complexifiant le déploiement.
- PyQt6 avec Matplotlib intégré : retenu. Permet une application desktop autonome, portable et richement interactive.

1.3 Hypothèses réductrices

Afin de rendre le projet réalisable dans les délais impartis et de conserver un périmètre pédagogique cohérent, plusieurs simplifications ont été introduites :

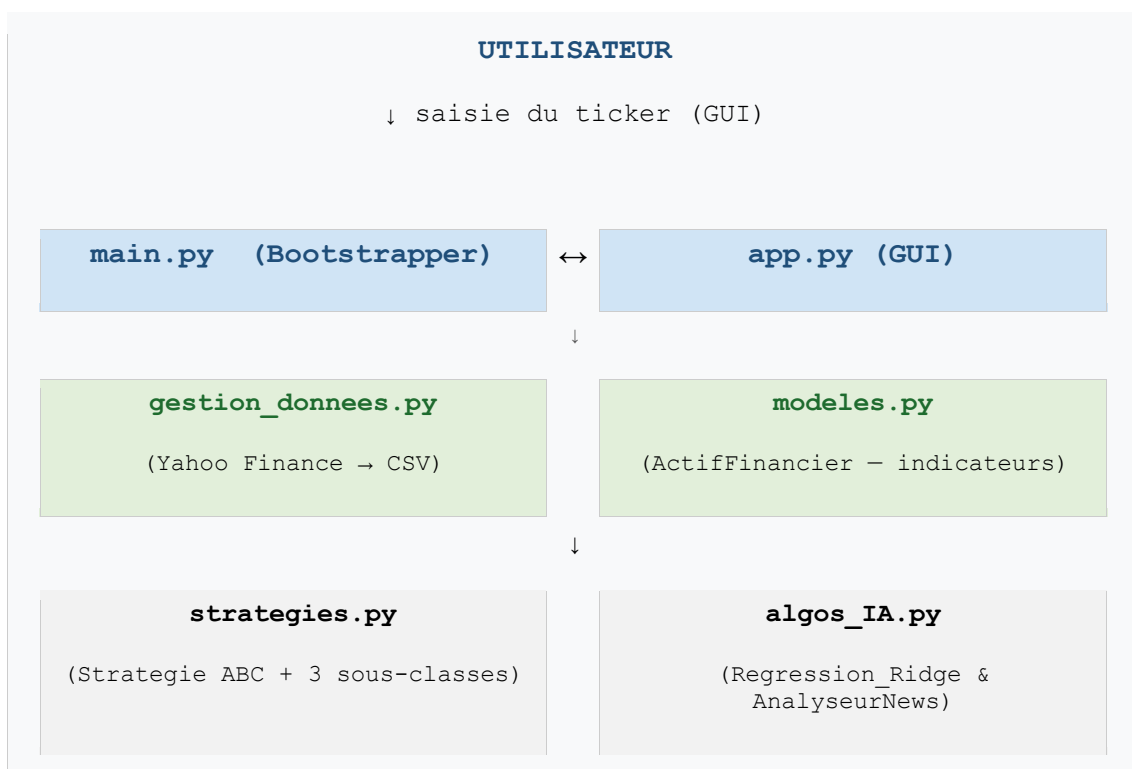
1. Long only : les stratégies ne prennent que des positions longues (achat). La vente à découvert n'est pas modélisée.
2. Absence de frais de transaction : le backtesting ne déduit ni commissions ni spreads. Les performances affichées sont donc théoriquement optimistes.
3. Pas de gestion du risque : pas de stop-loss, pas de dimensionnement de position (le modèle est toujours 100 % investi ou 0 %).
4. Horizon de prédiction limité à J+1 : l'IA prédit uniquement la direction du lendemain. Aucun signal moyen ou long terme n'est généré.
5. Données journalières uniquement : les données intraday (ticks, bougies 1 min) ne sont pas prises en compte.
6. Un seul actif à la fois : le screener analyse un ticker à la demande ; aucune liste d'univers n'est parcourue automatiquement.

Chapitre 2 — Description de l'application

2.1 Architecture générale

Le projet est composé de six modules Python indépendants (plus un fichier de tests). Le schéma ci-dessous illustre les dépendances et le flux d'exécution principal.

Figure 1 — Architecture modulaire du screener



Le flux d'exécution principal est entièrement événementiel : il est orchestré par l'interface graphique (le Contrôleur) et se déclenche à la saisie d'un ticker par l'utilisateur. Il se déroule en cinq étapes successives :

1. **Téléchargement et chargement des données (`gestion_donnees.py`) :**
récupération de 2 ans d'historique quotidien depuis Yahoo Finance et persistance en fichier CSV pour éviter les requêtes redondantes.
2. **Calcul des indicateurs techniques (`modeles.py`) :**
L'objet `ActifFinancier` enrichit son `DataFrame` en calculant à la volée les SMA, EMA, RSI, MACD, la volatilité historique et le Z-score du volume.
3. **Backtesting des stratégies (`strategies.py`) :**

Chaque stratégie rattachée à l'actif génère ses signaux sur l'historique complet et mesure son rendement théorique face au Buy & Hold de référence.

4. **Entraînement IA Technique et prédiction (algos_IA.py)** : préparation des données, *grid search* de l'hyperparamètre α , entraînement de la régression Ridge, évaluation des performances (R^2 , MAE, Hit Ratio) et prédiction directionnelle pour J+1.
5. **Analyse Fondamentale et Sémantique (algos_IA.py / modeles.py)** : l'IA AnalyseurNews effectue un scraping en temps réel des dernières actualités de l'actif, nettoie les textes, et calcule un score de sentiment et de confiance basé sur un lexique financier bilingue.

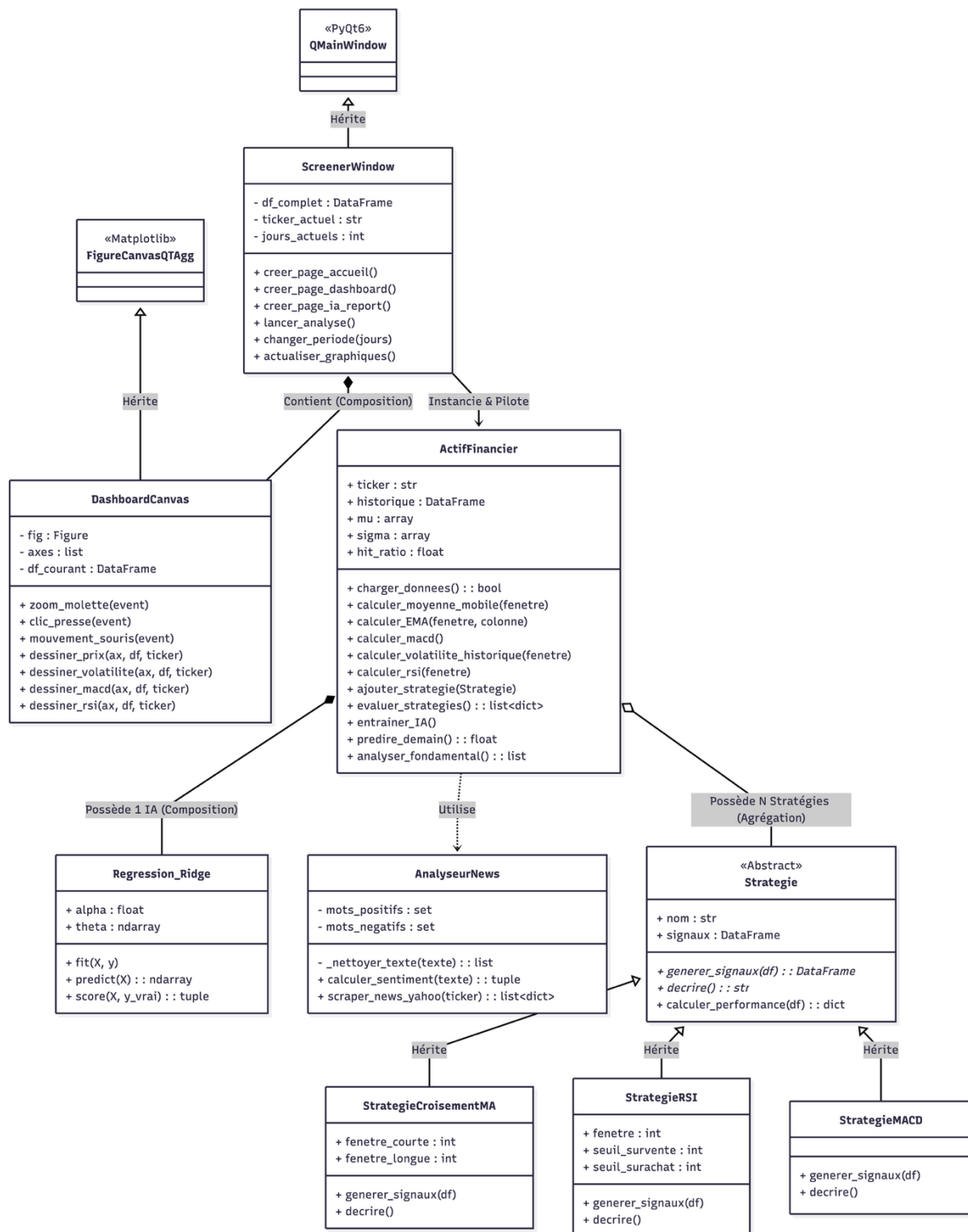
2.2 Principales classes et méthodes

Le tableau suivant recense les classes et modules essentiels du projet avec leur rôle respectif.

Classe / Module	Fichier	Rôle principal
ActifFinancier	modeles.py	Encapsule un actif boursier : historique, calcul des indicateurs techniques, gestion des stratégies (composition) et interface avec l'IA.
Strategie (ABC)	strategies.py	Classe abstraite définissant le contrat commun à toutes les stratégies : <code>generer_signaux()</code> , <code>decire()</code> , <code>calculer_performance()</code> (héritée).
StrategieCroisementMA	strategies.py	Implémentation concrète : génère des signaux sur Golden Cross / Death Cross de deux SMA paramétrables.
StrategieRSI	strategies.py	Implémentation concrète : détecte les sorties de zones de survente/surachat via le RSI calculé récursivement.
StrategieMACD	strategies.py	Implémentation concrète : croisement de la ligne MACD avec sa ligne signal (12/26/9).
Regression_Ridge	algos_IA.py	Régression Ridge implémentée manuellement (sans sklearn). Fournit

Classe / Module	Fichier	Rôle principal
		fit(), predict() et score() (R^2 , MAE, Hit Ratio).
AnalyseurNews	algos_IA.py	Scraper de News se basant sur l'API publique Yahoo et renvoyant un sentiment de marché (positif ou négatif). La logique repose sur une comparaison des mots extraits à deux dictionnaire prédéfinis de mots positifs et négatifs.
ScreeenerWindow	app.py	Fenêtre principale PyQt6 avec deux pages (accueil / dashboard) et graphiques interactifs via Matplotlib.
DashboardCanvas	app.py	Canvas Matplotlib intégré à PyQt6 ; gère zoom (molette), panoramique (glisser), et tooltip de survol.
telecharger_historique()	gestion_donnees.py	Télécharge 2 ans d'historique depuis Yahoo Finance et le persiste en CSV dans le dossier du projet.

Diagramme de Classe :



2.2.1 ActifFinancier (modeles.py)

C'est le nœud central du projet. Chaque instance représente un seul actif financier identifié par son ticker boursier. Ses attributs principaux sont :

- historique : DataFrame pandas contenant toutes les données brutes et les colonnes d'indicateurs calculées au fil des appels.
- strategies : liste Python (structure dynamique, figure 1) d'objets Strategie attachés à cet actif par composition.
- IA : instance de Regression_Ridge initialisée lors du grid search.
- mu / sigma : paramètres de standardisation calculés uniquement sur l'ensemble d'entraînement (80 % de l'historique).

La méthode `preparer_donnees_IA()` construit le vecteur de features normalisé (distance à la SMA, distance à l'EMA, volatilité, RSI, Z-score du volume) et le label cible (rendement du lendemain).

La séparation temporelle stricte **80/20 évite tout biais de look-ahead**.

La méthode `trouver_meilleur_alpha()` implémente un **grid search** sur sept valeurs de $\alpha \in \{0,001 ; 0,01 ; 0,1 ; 1 ; 10 ; 100 ; 1\,000\}$ en maximisant le Hit Ratio directionnel sur le jeu de test.

2.2.2 Hiérarchie Stratégie — héritage et polymorphisme

La classe abstraite Strategie (module ABC de Python) définit le contrat commun :

- `generer_signaux(df)` [méthode abstraite] : doit retourner un DataFrame avec une colonne Signal $\in \{-1, 0, +1\}$.
- `decrire()` [méthode abstraite] : retourne une description textuelle de la logique.
- `calculer_performance(df)` [méthode concrète héritée] : backteste la stratégie et retourne un dictionnaire de résultats.

*La méthode calcule également deux indicateurs de risque : le **Sharpe Ratio annualisé** (rendement moyen divisé par la volatilité des rendements, multiplié par $\sqrt{252}$), qui mesure le rendement par unité de risque ; et le **Max Drawdown** (pire chute depuis un sommet), calculé séparément pour la stratégie et pour le Buy & Hold de référence, exprimé en pourcentage.*

Les trois sous-classes concrètes surchargent exclusivement les deux méthodes abstraites :

- StrategieCroisementMA : Golden Cross (SMA courte > SMA longue) → signal +1 ; Death Cross → signal -1.
- StrategieRSI : sortie de zone de survente (RSI remonte au-dessus de 30) → +1 ; sortie de surachat (RSI repasse sous 70) → -1.
- StrategieMACD : croisement haussier MACD / ligne signal → +1 ; croisement baissier → -1.

Le polymorphisme s'exprime dans `evaluer_strategies()` : la boucle appelle `s.calculer_performance()` sur chaque élément de la liste, indépendamment de sa classe concrète réelle.

2.2.3 Regression_Ridge (algos_IA.py)

L'implémentation est entièrement manuelle. La méthode `fit()` résout directement la forme analytique :

$$\theta = (X^T X + \alpha I)^{-1} X^T y$$

où I est la matrice identité dont le premier coefficient diagonal est mis à zéro afin de ne pas pénaliser le terme de biais. La méthode `score()` calcule trois métriques :

- R^2 (coefficient de détermination) : mesure la part de variance expliquée. En finance, un R^2 proche de 0 est normal.
- MAE (Mean Absolute Error) : erreur absolue moyenne exprimée en pourcentage de rendement.
- Hit Ratio (accuracy directionnelle) : proportion de jours où le signe de la prédiction correspond au signe réel. C'est la métrique opérationnelle la plus pertinente pour un modèle de trading.

La régression de Ridge vise à minimiser la fonction cout :

$$J_{Ridge}(\theta) = \sum_{i=1}^m (y_i - X_i \theta)^2 + \alpha \sum_{j=1}^n \theta_j^2$$

Dans le fichier `Algos_IA.py`, l'utilisation de "`I_modifié`" (où le premier coefficient est nul) permet de ne pas pénaliser le biais (l'ordonnée à l'origine), garantissant que le modèle peut s'ajuster au niveau moyen des prix sans contrainte après

ajout du `np.column_stack` qui permet de multiplier l'ordonnée à l'origine par 1.

La régression Ridge reste un modèle linéaire. Les marchés financiers comportant de fortes non-linéarités, le modèle capte davantage des corrélations simples que des schémas complexes de marché.

2.2.4 Indicateurs pris en compte

EMA : Exponential moving average

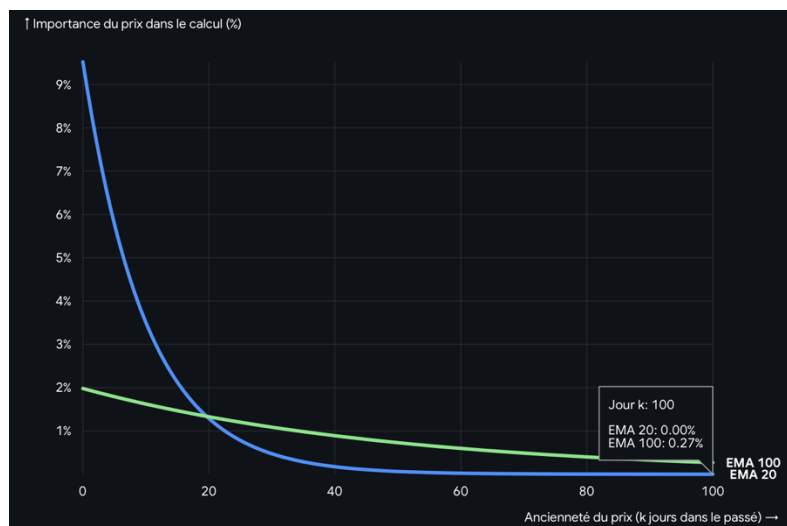
$$EMA_t = \alpha \cdot P_t + (1 - \alpha) \cdot EMA_{t-1}$$

Où :

- P_t est le prix d'aujourd'hui.
- EMA_{t-1} est la valeur de l'EMA d'hier.
- Alpha est le multiplicateur (ou coefficient de lissage).

L'impact d'un prix passé (il y a k jours) diminue selon le facteur $(1-\alpha)^k$.

- Avec l'**EMA 20**, ce facteur est d'environ 0,90k. L'importance des prix passés s'effondre très vite (au bout de 20 jours, le prix d'il y a 20 jours ne pèse presque plus rien).
- Avec l'**EMA 100**, ce facteur est d'environ 0,98k. La mémoire de l'indicateur est beaucoup plus longue



RSI :

Le **RSI (Relative Strength Index)**, ou Indice de Force Relative, est un oscillateur de momentum développé par J. Welles Wilder en 1978. Il permet de mesurer la vitesse

et l'amplitude des variations de prix afin d'évaluer si un actif est "suracheté" (potentielle correction à la baisse) ou "survendu" (potentiel rebond à la hausse).

Le calcul se fait en deux étapes.

1. Le calcul de la Force Relative (RS)

On calcule d'abord le rapport entre la moyenne mobile exponentielle (EMA) des gains et la moyenne des pertes en valeur absolue.

$$RS = \frac{\text{Moyenne des gains}}{\text{Moyenne des pertes}}$$

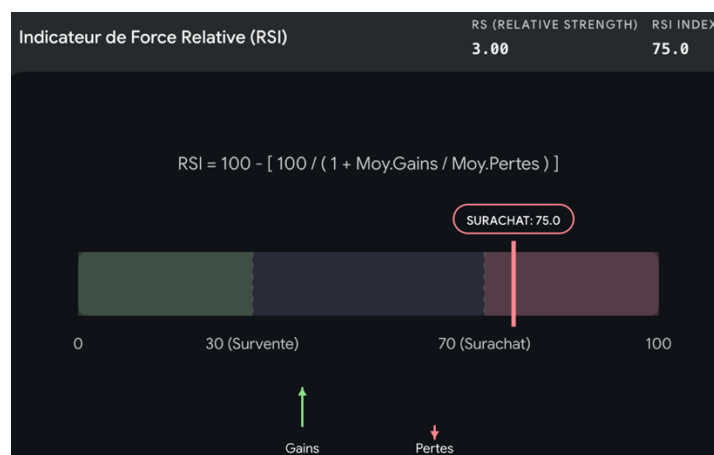
2. La normalisation en RSI

Pour que l'indicateur soit facilement lisible, le RS est transformé via une formule mathématique qui le contraint (le normalise) dans un couloir strict compris entre 0 et 100 :

$$RSI = 100 - \frac{100}{1 + RS}$$

Interprétation mathématique :

- Si les pertes moyennes sont nulles (que des hausses), le dénominateur de la fraction tend vers l'infini, la fraction vaut 0, donc le RSI =100.
- Si les gains moyens sont nuls (que des baisses) RS = 0, la fraction vaut 100, donc le RSI = 0 .
- Les seuils standards d'alerte sont à **30** (survente) et **70** (surachat).



Volatilité (Historique) :

La volatilité historique, dans le contexte des marchés financiers, est une mesure statistique de la dispersion des rendements d'un actif. Plus simplement, elle quantifie le "risque" ou la "nervosité" d'une action : un actif très volatil connaîtra de fortes variations de prix à la hausse comme à la baisse, tandis qu'un actif peu volatil aura un cours plus régulier. Le calcul repose sur le calcul de la variance puis de l'écart type annualisé en multipliant par racine de 252 (le nombre de jours ou la bourse est ouverte).

Z-score du Volume :

On prend le volume du jour V_t , on lui soustrait la moyenne simple sur 20 jours, et on divise le tout par l'écart-type sur 20 jours.

$$Z_t = \frac{V_t - \mu_{20}}{\sigma_{20}}$$

2.2.5 Fonction récursive — calcul de l'EMA de Wilder

La méthode `calculer_EMA_recuratif()` (présente dans `StrategieRSI` et dans `ActifFinancier`) calcule récursivement la moyenne exponentielle de Wilder, utilisée dans le calcul du RSI :

- Cas de base : $\text{EMA}(0) = \text{valeurs}[0]$
- Cas récursif : $\text{EMA}(i) = \text{valeurs}[i] \times \alpha + \text{EMA}(i-1) \times (1 - \alpha)$,

Avec $\alpha = 1 / \text{fen\^etre}$.

La méthode récursive `calculer_EMA_RSI_recuratif()` existe dans `ActifFinancier` (et dans `StrategieRSI`) et implémente correctement le lissage de Wilder par récursion terminale. Elle satisfait la figure imposée. En pratique, `calculer_rsi()` dans `ActifFinancier` utilise une boucle itérative équivalente pour éviter la limite de récursion Python (qui plante sur des séries de 500+ points avec une récursion $O(n^2)$). Les deux approches coexistent : la récursive prouve la maîtrise du concept, l'itérative est l'implémentation de production.

2.2.6 News Scraper (algorithms IA.py)

L'implémentation de cette analyse sémantique a été réalisée *from scratch*, sans faire appel à des bibliothèques de haut niveau de type `yfinance`. Le module effectue des requêtes HTTP natives (`urllib`) vers l'API interne de recherche de Yahoo Finance.

L'algorithme nettoie ensuite les données brutes (ponctuation, casse) et calcule une « densité sémantique ». Ce score de confiance est obtenu en évaluant le ratio des mots identifiés grâce à un lexique financier bilingue (positif/négatif) sur le nombre total de mots du titre.

2.2.7 Interface graphique (app.py)

L'interface repose sur deux classes PyQt6 :

- ScreenerWindow (QMainWindow) : fenêtre principale à deux pages gérées par un QStackedWidget. La page d'accueil contient un champ de saisie du ticker ; la page dashboard affiche les graphiques et la prédiction IA. Des boutons permettent de filtrer la fenêtre temporelle (1 mois, 3 mois, 6 mois, 1 an, 2 ans, Max).
- DashboardCanvas (FigureCanvas) : canvas Matplotlib embarqué dans Qt, subdivise la figure en quatre sous-graphiques (prix/bougies + SMA/EMA, volatilité, MACD, RSI). Il gère le zoom par molette, le panoramique par glisser-déposer et un tooltip flottant qui affiche les valeurs OHLC et les indicateurs au passage de la souris.

2.2.8 Lancement du programme

Après installation des dépendances, le programme se lance depuis le répertoire du projet via :

```
pip install yfinance pandas numpy matplotlib PyQt6  
  
python main.py
```

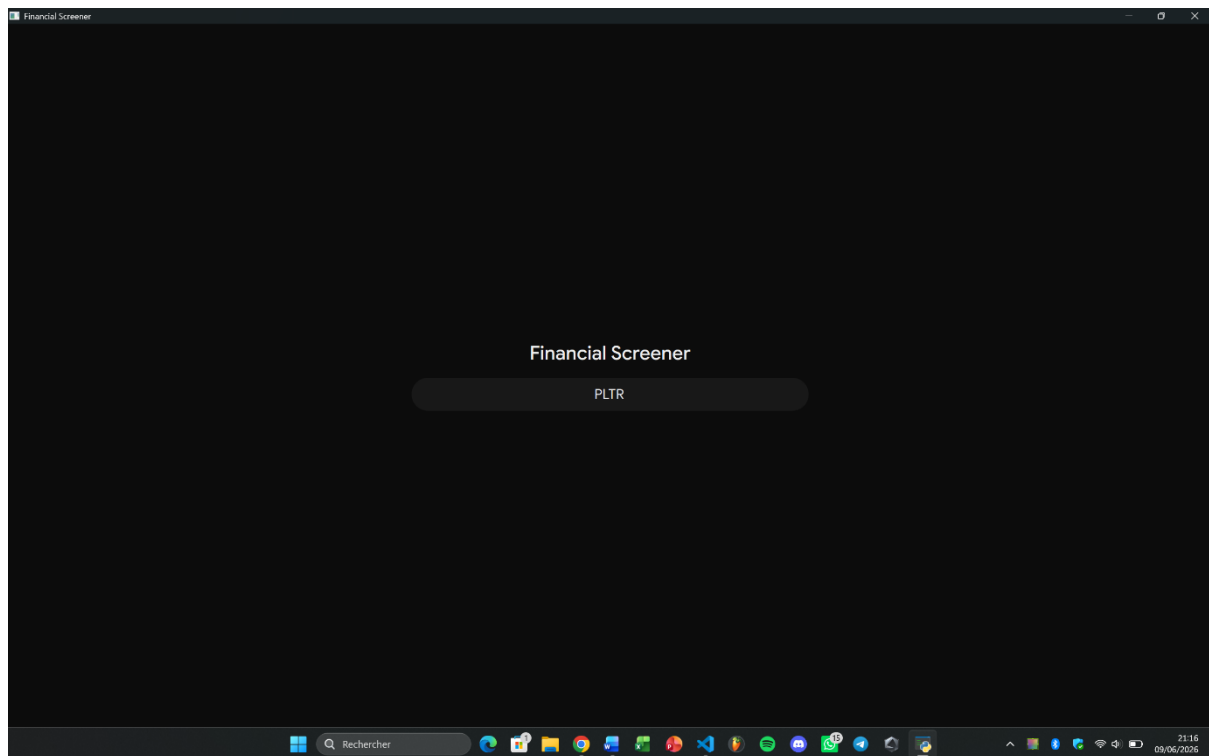
Le programme ouvre automatiquement la fenêtre graphique.

2.3 Exemple d'utilisation

Ci-joint quelques captures d'écran montrant directement comment l'application est utilisée.

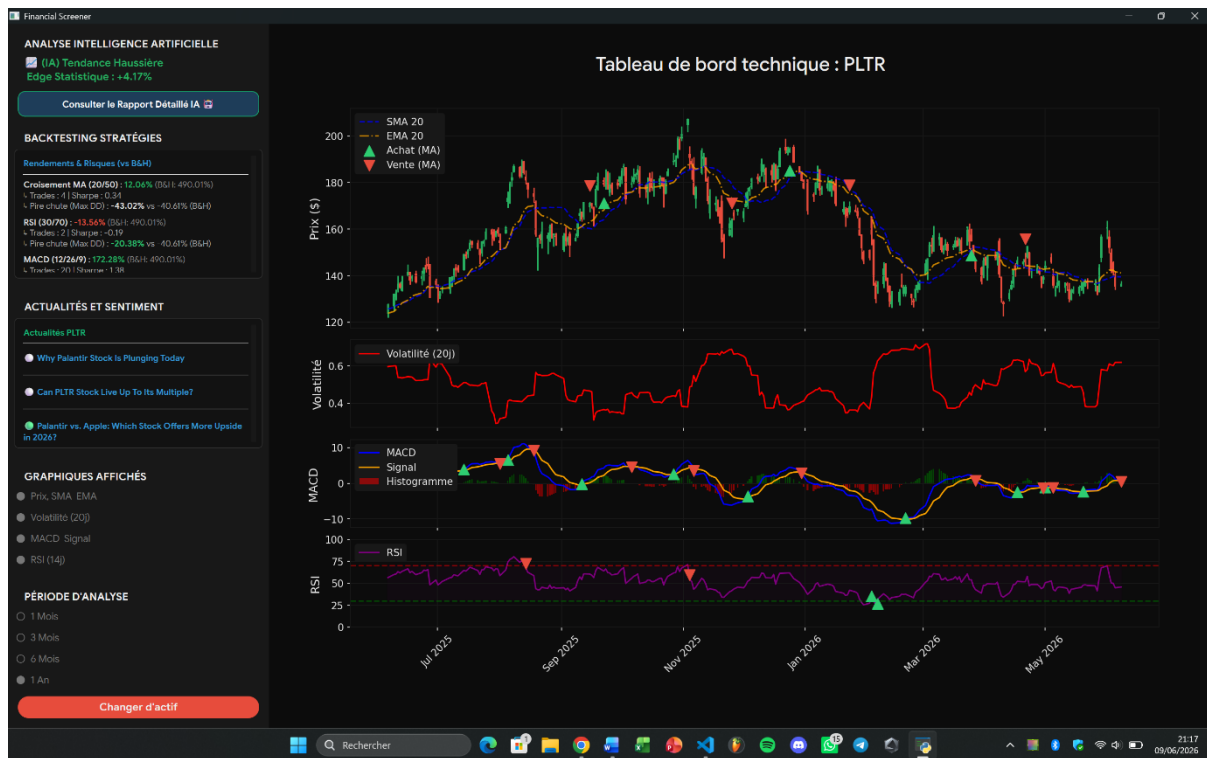
2.3.1 Page d'accueil

Ce sur quoi on tombe après exécution du main. On écrit le ticker (l'abréviation de l'entreprise cotée en bourse) dans la boîte de recherche, afin de faire comprendre au programme qu'il doit télécharger les informations relatives à l'entreprise et faire les calculs nécessaires pour l'analyse de cette dernière.

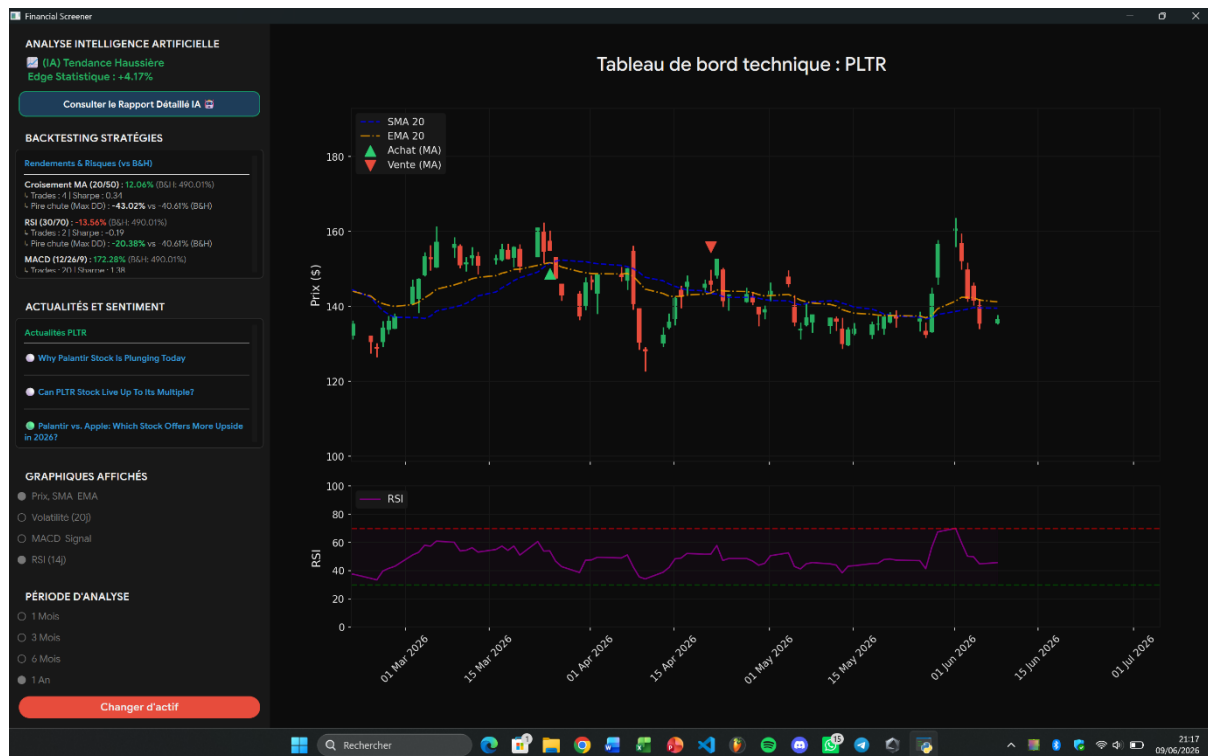


2.3.2 Page principale

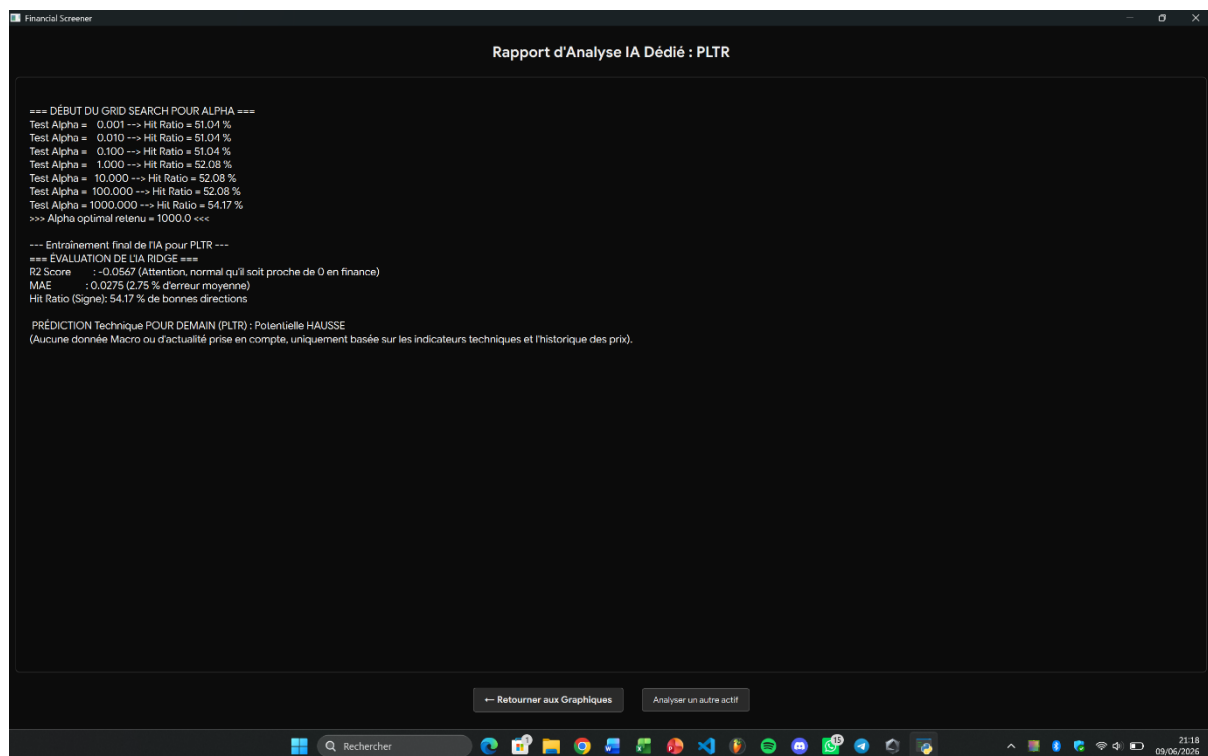
Là où se trouvent toutes les informations financières relatives à l'entreprise. On y a plusieurs données représentées sur différents graphiques (prix, SMA, EMA, volatilité, MACD, RSI) qu'on peut choisir ou non d'afficher pour plus de lisibilité. On y a également une section sur l'actualité relative à l'entreprise, une sur le backtesting des stratégies, et un rapport détaillé de l'IA, montrant l'ensemble des calculs intermédiaires permettant d'aboutir à la prédiction de l'évolution du prix de l'action (prédiction donnée tout en haut à gauche).



Ici, on a par exemple choisi de se concentrer uniquement sur le prix et sur le RSI de l'action, tout en zoomant sur son évolution récente.



Ici le rapport détaillé de l'IA.



Chapitre 3 — Conclusion

3.1 Tableau récapitulatif des figures imposées

Le tableau ci-dessous recense les sept figures imposées et leur implémentation dans le projet.

N°	Figure imposée	Fichier	Emplacement / Remarque
1	Utilisation d'une structure de données dynamique	modeles.py	ActifFinancier.strategies → liste Python (list) qui grandit par append()
2	Utilisation d'un dictionnaire	strategies.py / modeles.py / algos_IA.py	calculer_performance() retourne un dict ; evaluer_strategies() retourne list[dict]
3	Héritage entre types créés par l'étudiant	strategies.py	Strategie (ABC) ← StrategieCroisementMA, StrategieRSI, StrategieMACD
4	Polymorphisme / méthodes abstraites	strategies.py	generer_signaux() et decrire() sont abstraites dans Strategie, surchargées dans chaque sous-classe
5	Tests unitaires (≥ 4 méthodes, ≥ 2 cas/méthode)	tests_unitaires.py	30 méthodes de test, 4 classes TestCase : TestRegressionRidge, TestActifFinancier, TestStrategies
6	Lecture / écriture de fichiers	gestion_donnees.py / modeles.py	telecharger_historique() → CSV (écriture) ; charger_donnees() → CSV (lecture via pandas)
7	Fonction récursive dans le cœur du projet	strategies.py / modeles.py	StrategieRSI.calculer_EMA_recuratif() et ActifFinancier.calculer_EMA_RSI_recuratif() : récursion terminale pour le lissage de Wilder

3.2 Tests effectués

Le fichier `tests_unitaires.py` contient 30 méthodes de test couvrant 25 comportements clés (fonctionnels et gestion d'erreurs), détaillés dans le tableau suivant :

Exécution : `python -m unittest tests_unitaires -v`

N°	Classe testée	Méthode / comportement	Résultat attendu
1	TestRegressionRidge	predict() avant fit()	Lève ValueError
2	TestRegressionRidge	predict() après fit() — forme de sortie	shape[0] == len(X)
3	TestRegressionRidge	R ² sur données quasi-linéaires	R ² ≥ 0,99
4	TestRegressionRidge	Dégradation R ² avec fort alpha	R ² _fort < R ² _faible
5	TestRegressionRidge	Hit Ratio borné [0, 1]	0 ≤ hit ≤ 1
6	TestRegressionRidge	Hit Ratio sur données à fort signal	hit > 0,85
7	TestActifFinancier	SMA — NaN au début (fenêtre=20)	19 premières valeurs NaN
8	TestActifFinancier	SMA — valeur exacte sur série connue	Moyenne exacte à 5 décimales
9	TestActifFinancier	RSI — bornes [0, 100]	Toutes valeurs ∈ [0, 100]
10	TestActifFinancier	RSI — trend haussier → RSI > 90	RSI final > 90
11	TestActifFinancier	Volatilité — dépendance aux rendements	Colonne absente si Rendements manquent
12	TestActifFinancier	Volatilité — toujours positive ou nulle	vol ≥ 0
13	TestActifFinancier	charger_donnees() ticker inexistant	Retourne False

N°	Classe testée	Méthode / comportement	Résultat attendu
14	TestActifFinancier	charger_donnees() injection directe	historique non vide
15	TestStrategies	Signaux CroisementMA $\in \{-1, 0, 1\}$	Pas de valeur hors ensemble
16	TestStrategies	Signaux RSI $\in \{-1, 0, 1\}$	Pas de valeur hors ensemble
17	TestStrategies	calculer_performance() — clés retournées	Les 4 clés présentes
18	TestStrategies	Nombre de trades ≥ 0	int ≥ 0
19	TestStrategies	MACD sans colonnes → signaux neutres	Tous signaux = 0
20	TestStrategies	MACD avec colonnes → signaux non nuls	Au moins 1 signal $\neq 0$
21	TestStrategies	isinstance — héritage CroisementMA	True
22	TestStrategies	isinstance — héritage RSI	True
23	TestAnalyseurNews	Nettoyage de texte	Ponctuation retirée, texte converti en minuscules sans erreurs
24	TestAnalyseurNews	Calcul de sentiment	Les phrases avec mots clés renvoient le sentiment (Positif/Négatif) et un score > 0
25	TestAnalyseurNews	Scraping réseau (Mocking)	Si Yahoo renvoie une erreur réseau, l'interface gère l'exception et affiche un message d'erreur — le scraper remonte l'exception au lieu de l'avalier.

L'ensemble des 25 cas de test s'exécute avec succès sur des données synthétiques générées localement. Conformément aux exigences d'isolation des tests unitaires, toutes les méthodes nécessitant habituellement une connexion internet (qu'il s'agisse du téléchargement des cours ou du scraping des actualités financières) font l'objet d'un *Mocking* via `unittest.mock.patch`. Cette approche permet de simuler le comportement des serveurs distants et la gestion des erreurs réseau en circuit fermé, garantissant une exécution du jeu de tests à 100 % hors ligne, stable et reproductible.

3.3 Limitations identifiées

- Performance IA Ridge modeste :

Le Hit Ratio directionnel (indiquant si l'IA prédit bien une hausse ou une baisse) se situe typiquement entre 50 % et 56 % sur les données de test, ce qui est légèrement supérieur au hasard mais insuffisant pour une utilisation en conditions réelles sans gestion du risque.

R2 souvent très proche de 0 : Normal pour des marchés financiers aléatoires avec distribution de probabilité Indépendantes et identiquement distribués.

MAE : autour de 1%, élevé mais cohérent pour notre cas d'étude lié aux marchés financiers.

- **Absence de frais** et de slippage : les backtests surestiment les rendements réels. L'intégration de modèles de coûts de transaction rendrait les comparaisons plus réalistes.
- **Données journalières uniquement** : les signaux générés ne sont pas utilisables pour du trading intraday.
- Profondeur limitée à 2 ans : yfinance permet de récupérer des historiques plus longs. Augmenter la fenêtre améliorerait peut être la robustesse de l'entraînement IA.
- Pas de diversification : l'outil analyse un actif à la fois. Une extension naturelle serait l'analyse d'un univers d'actifs avec filtrage multi-critères.
- Programme couteux par moments (Récursion EMA coûteuse) : la fonction `calculer_EMA_recuratif()` a une complexité $O(n^2)$ car elle repart de zéro pour chaque point. Une implémentation itérative serait bien plus efficace sur des séries longues.

3.4 Apports

- **Grid Search de l'hyperparamètre α** : la sélection automatique du meilleur coefficient de régularisation via validation sur données de test n'était pas imposée mais améliore significativement la qualité des prédictions.
- **Standardisation rigoureuse (data leakage)** : les paramètres de normalisation (μ , σ) sont calculés exclusivement sur l'ensemble d'entraînement et appliqués au jeu de test, conformément aux bonnes pratiques de l'apprentissage automatique.
- **Interface graphique avec bougies japonaises** : l'affichage en chandeliers (candlesticks) avec distinction colorée des jours haussiers/baissiers va au-delà d'un simple graphique de cours de clôture.
- **Zoom et panoramique interactifs** : la gestion des événements souris dans DashboardCanvas (molette, glisser, tooltip) enrichit l'expérience utilisateur sans bibliothèque externe.
- **Trois métriques d'évaluation IA** : l'affichage conjoint du R^2 , de la MAE et du Hit Ratio permet une lecture nuancée des performances, en distinguant l'erreur quantitative de l'accuracy directionnelle.
- **Métriques de risque dans le backtesting** : au-delà du simple rendement, chaque stratégie est évaluée sur son Sharpe Ratio annualisé et son Max Drawdown, permettant une comparaison risque/rendement rigoureuse face au Buy & Hold. Cette double lecture (performance brute vs performance ajustée au risque) n'était pas imposée et constitue un apport significatif.

3.5 Perspectives et améliorations

3.5.1 Améliorations algorithmiques

- Intégrer des modèles non linéaires (arbres de décision, gradient boosting) pour capturer des relations plus complexes entre les indicateurs.
- Ajouter plus de features macro avec un parser Internet au-delà des actualités financières (taux d'intérêt, VIX)
- Implémenter un mécanisme de re-entraînement incrémental (walk-forward optimization) pour maintenir la pertinence du modèle dans le temps.
- Passer à une régression logistique ou un classifieur binaire (hausse/baisse) plutôt qu'une régression du rendement, ce qui correspondrait mieux à l'objectif directionnel.

3.5.2 Améliorations fonctionnelles

- Screener multi-actifs : analyser automatiquement un univers (par exemple le CAC 40 ou le S&P 500) et classer les actifs par signal IA ou force de tendance.
- Backtesting avec frais : intégrer un modèle de coûts de transaction (commission fixe + spread variable) pour obtenir des rendements nets plus réalistes.
- Alertes en temps réel : surveiller les cours en continu (API streaming) et déclencher des notifications lorsque des signaux forts sont détectés.
- Export des résultats : sauvegarder les graphiques en PNG/PDF et les tableaux de performance en CSV ou en rapport HTML.

3.5.3 Améliorations techniques

- Remplacer la récursion par une implémentation itérative pour l'EMA de Wilder, réduisant la complexité de $O(n^2)$ à $O(n)$.
- Utiliser QThread (PyQt6) pour exécuter l'entraînement IA en arrière-plan et éviter le gel de l'interface graphique.

— Fin du rapport —